WelfareOne

청년 복지정책 맞춤 추천·안내 AI — 동작 설명서

기준: `feature/jimin-search-ux` 브랜치 · 2026-10-08
프론트엔드·백엔드·AI 호출이 어떻게 맞물려 동작하는지 팀원용으로 정리했습니다.

목차

1. 서비스가 하는 일
2. 사용하는 기술(도구)
3. 전체 구조와 폴더
4. 사용자 화면 흐름
5. API 목록과 호출 시점
6. 질문 분석 (route·intent)
7. 검색이 동작하는 순서
8. AI 관련도 재판정과 후보 표시
9. 후속 질문과 답변 (자주 묻는 질문 버튼)
10. API 키(BYOK)·보안·오류 처리
11. 브라우저에 저장되는 데이터
12. 데이터(DB) 만드는 과정
13. 실행·설정·테스트
14. 비용·지연, 한계와 다음 작업

부록. 용어 풀이

1. 서비스가 하는 일

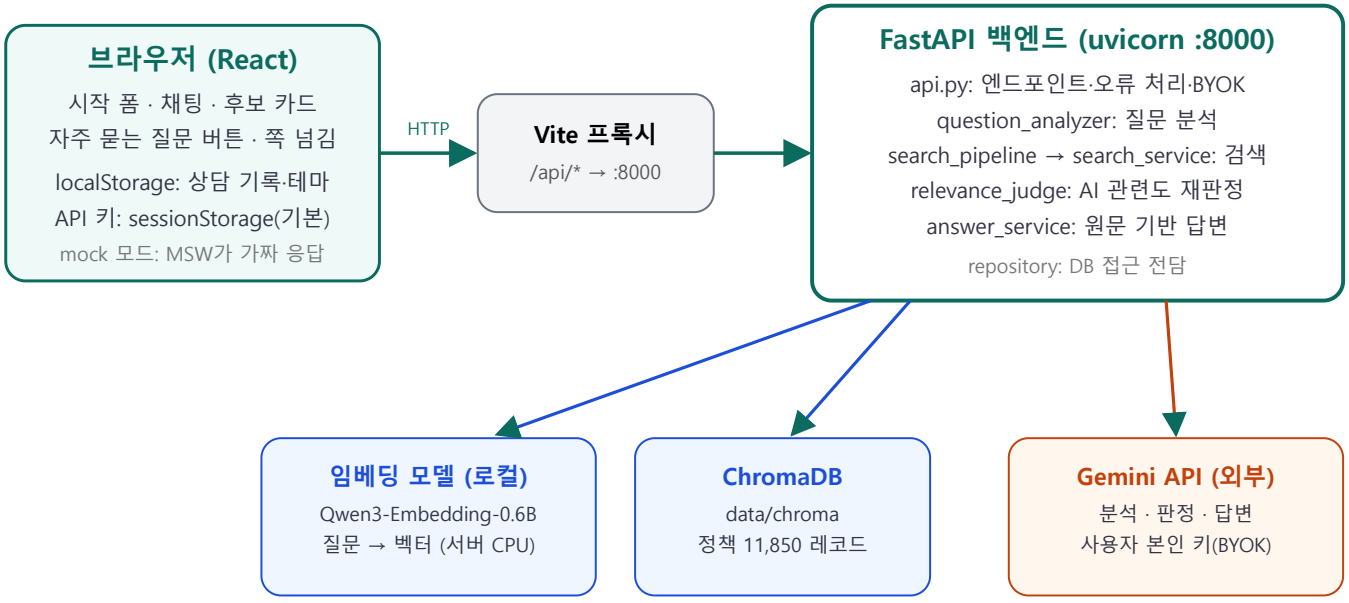
사용자가 **지역·나이**와 **자연어 질문**("월세가 부담돼요", "배고파")을 입력하면 정책 후보를 찾아 보여 주고, 사용자가 고른 정책의 **공식 원문에 근거해** 질문에 답하는 웹 서비스입니다. 자격 충족을 확정하지 않는 **예비 안내**이며 최종 확인은 해당 기관에서 합니다.

1. **시작** — 시·도, 시·군·구, 나이(선택)를 입력합니다.
2. **질문** — 상황이나 정책 이름을 자유롭게 입력합니다(예시 질문 버튼도 있음).
3. **후보 확인** — "내 지역 정책"과 "중앙부처 정책"이 따로 나옵니다. 관련 있는 후보가 먼저, 관련도 낮은 후보는 배지와 함께 뒤에 이어지며 3개씩 이전/다음으로 넘겨 봅니다.
4. **정책 선택** — 카드를 누르면 "현재 정책"이 되고 자주 묻는 질문 버튼 6개가 나옵니다.
5. **후속 질문** — 버튼(지원 대상·지원 내용·신청 방법·신청 기간·필요 서류·문의처)을 누르거나 직접 입력합니다.
6. **답변** — 소제목·불릿으로 정리된 답변과 근거 구절, 첨부파일·누리집 링크가 나옵니다. "원문 보기", "다른 정책 찾기", "새 상담"으로 이어갑니다.

2. 사용하는 기술(도구)

영역	도구	역할
프론트엔드	React 18.3 + TypeScript 5.6, Vite 5.4, react-markdown 9	채팅 화면, 후보 카드, 답변 말풍선(마크다운). Vite 개발 서버가 `/api/*`를 백엔드로 중계(프록시)해 CORS 설정이 필요 없음
프론트 mock	MSW 2.6 (Mock Service Worker)	백엔드·키 없이 화면 개발. 질문에 `더보기`·`관련낮음`·`재판정` 같은 키워드를 넣어 상황 재현
백엔드	Python(3.11 이상, 개발 PC 3.14), FastAPI 0.141, uvicorn 0.52	REST API. 모델·DB는 서버 시작 때 한 번만 로드. Gemini 호출이 동기라 엔드포인트는 `def`로 두어 별도 스레드에서 실행
벡터 DB	ChromaDB 1.5.9 (`data/chroma`)	컬렉션 `welfare_policy_sections_qwen_v3`, 11,850 레코드(정책 1,418개 = 지자체 1,253 + 중앙 165), 코사인 거리. 저장소에는 올리지 않음
임베딩 모델	sentence-transformers 6.0.1 + torch 2.14, `Qwen/Qwen3-Embedding-0.6B`	질문을 숫자 벡터로 바꿔 DB에서 비슷한 정책을 찾음. **서버 PC에서 로컬 실행**(CPU), 이미 내려받은 모델만 사용 (`local_files_only`)
LLM	Gemini (google-genai 2.24), 기본 모델 `gemini-3.5-flash-lite`	질문 분석, 관련도 판정, 원문 기반 답변 생성. 모델은 환경변수 `GEMINI_MODEL`로 변경
데이터	복지로 공공 API(XML) → JSONL → ChromaDB	`scripts/`의 수집·변환·적재 스크립트(12절)
테스트	unittest(백엔드 131개), node:test + esbuild(프론트 39개)	실제 DB 스모크 테스트는 `WELFAREONE_REAL_TESTS=1`일 때만 실행

3. 전체 구조와 폴더



파란색 = 서버 PC 안에서 처리(무료), 주황색 = 외부 호출(키·한도·비용)

폴더 구조

폴더	내용
`app/`	백엔드. `api.py` (FastAPI), `cli.py` (터미널 프로토타입), 검색·분석·답변·판정 서비스
`frontend/src/`	`api/` 요청, `features/` 화면(start·chat·policy·settings), `state/` 대화 상태, `storage/` 상담 기록, `utils/`, `mocks/`, `types/contract.ts` (API 타입)
`docs/`	계약서(`function_contracts.md`), 계약서 수정 제안, 변경 요약, 설계 초안, 라벨링 가이드
`scripts/`	데이터 수집·적재, 거리 측정, 라벨링 시트·Reranker 실험 스크립트
`data/`	`raw` (원본 XML), `processed` (JSONL), `evaluation` (평가 데이터), `chroma` (벡터 DB, Git 제외)
`tests/`	백엔드 단위 테스트, 평가 노트북

백엔드 모듈 역할

파일	역할
`api.py`	엔드포인트 4종 + `/health`, 사용자 키(BYOK) 처리, 오류를 계약서 코드로 변환
`search_pipeline.py`	"분석 결과 → 검색 → (선택) AI 재판정 → 실패 시 거리순 대체"를 한곳에서 실행. API와 CLI가 함께 사용
`search_service.py`	질문 임베딩, 지역별 DB 조회, 점수 계산, 거리 하한, 나이 필터
`search_profiles.py`	질문 유형(route)별 검색 설정(상황형은 하한 0.55, 지역 가산점 0)
`relevance_judge.py`	Gemini로 후보 관련도를 한 번에 판정하고 관련 → 관련도 낮음 순으로 재구성
`question_analyzer.py`	Gemini로 질문의 route·intent·검색어·키워드 분석(구어체를 정책 용어로 번역)
`answer_service.py`	선택 정책의 원문 섹션만 모아 Gemini로 답변 생성, 근거·링크·grounded 처리, 질문 유형별 답변 구성
`repository.py`	ChromaDB 접근 전담(지역 목록, 벡터 조회, 정책 섹션·메타 조회)
`gemini_retry.py`, `env.py`	Gemini 재시도 규칙(429·5xx, 1초·2초 대기), `.env` 읽기
`models.py`, `config.py`, `main.py`	공통 모델·설정·터미널 진입점. **팀 협의 없이 수정 금지**

4. 사용자 화면 흐름

화면/기능	동작
시작 폼	시·도 → 시·군·구(없는 지역은 비활성) 선택, 나이 0~120(비워도 됨). 지역 목록은 시작할 때 `GET /regions`로 받음
채팅	질문 입력 → 로딩 말풍선 → 결과. 대화 상태는 `useReducer`로 관리하며 계약서의 `ConversationState` 모양을 따름
후보 카드	정책명, 지역, 요약 한 줄, 범위 배지. 지자체/중앙부처를 섞지 않고 그룹별로 표시. 관련 후보가 먼저, 관련도 낮음(점선+배지)이 뒤에 이어지며 그룹마다 3개씩 이전/다음. 3개 이하면 버튼 없음
정책 선택	"현재 정책" 바 표시(원문 보기·다른 정책 찾기). 선택 안내 메시지와 이 바에 자주 묻는 질문 버튼 6개
답변 말풍선	마크다운 렌더링(원시 HTML 허용 안 함), 근거 접기, 첨부파일 다운로드/바로가기 버튼, 근거가 없으면 경고 표시
원문 보기	선택 정책의 섹션별 원문을 그대로 표시(`GET /policies/{id}`)
상담 목록(왼쪽)	날짜별 그룹, 전환·삭제·전체 삭제, ✎로 제목 수정(Enter 저장, Esc 취소, 비우면 첫 질문으로 복원, 최대 40자). 응답 대기 중에는 전환·삭제 불가
테마·API 키	라이트/다크/시스템 테마, "API 키" 버튼으로 본인 Gemini 키 입력

5. API 목록과 호출 시점

Method	URL	역할	Gemini 호출
GET	`/health`	서버 준비 확인 → `{"status":"ok"}`	없음
GET	`/regions`	시·도별 시·군·구 목록	없음
POST	`/policies/search`	질문 → 분석 → 검색 → 후보(지자체/중앙) + 판정 결과	1회(분석) + 1회(재판정, 상황형일 때)
GET	`/policies/{policy_id}`	정책명·지역·섹션별 원문	없음
POST	`/policies/{policy_id}/ask`	선택 정책에 대한 질문 → 원문 기반 답변	버튼 질문 1회, 직접 입력 2회

검색 결과가 없는 것은 오류가 아니라 **200 + 빈 목록**입니다. `region.level2`가 없으면 `null`이 아니라 `""`(DB가 광역 정책을 빈 문자열로 저장). 서버는 상태가 없는(stateless) 구조라 지역·나이·선택 정책은 프론트가 들고 있다가 요청마다 보냅니다.

어떤 경우에 API를 호출하나 (프론트 동작 기준)

사용자 행동	호출	서버 안에서 일어나는 일
앱 시작(시작 폼)	`GET /regions`	시작할 때 한 번 읽어 둔 지역 목록 반환(DB 조회 없음)
질문 입력(정책 선택 전)	`POST /policies/search`	Gemini 질문 분석 → 로컬 임베딩 → DB 지역별 조회 → Gemini 관련도 재판정(상황형 질문, 기본 켜짐) → 응답
후보 카드 선택	없음	프론트 상태만 바뀜(현재 정책 지정, 안내 메시지 + 버튼 표시)
이전/다음 쪽 넘김, 관련도 낮은 후보 보기	없음	이미 받은 후보(그룹당 최대 12개)를 화면에서 3개씩 보여줌
"원문 보기"	`GET /policies/{id}`	DB에서 섹션별 원문 조회
자주 묻는 질문 버튼	`POST /policies/{id}/ask` + `intent`	분석 생략 → 해당 유형의 섹션 조회 → Gemini 답변 1회
직접 입력(정책 선택 후)	`POST /policies/{id}/ask`	Gemini 질문 분석 → 현재 정책 질문이면 Gemini 답변, 아니면 답변 없이 "다른 정책 찾기" 안내
다른 정책 찾기 / 새 상담	없음	프론트 상태 초기화(상담 기록은 브라우저에 남음)
상담 전환·삭제·제목 수정	없음	모두 브라우저 localStorage 안에서 처리

프론트 요청 제한 시간은 30초입니다(Gemini 재시도 최대 3회를 고려). 터미널 프로토타입(`python -m app.main`)은 같은 서비스를 직접 호출하며 AI 재판정은 하지 않습니다.

6. 질문 분석 (route·intent)

질문이 들어오면 Gemini가 아래 값을 JSON으로 돌려주고, 코드가 허용값·자료형을 검증합니다. 현재 정책이 없는데 "후속 질문"으로 분류되면 코드가 `situation_search` 또는 `specific_policy`로 고칩니다. 분석은 `seed=0`, `temperature=0`으로 같은 질문에 같은 결과가 나오게 합니다.

route	의미 / 이후 처리
`situation_search`	상황을 말하며 추천 요청("월세가 부담돼요"). 엄격한 검색 설정 + AI 재판정 적용
`specific_policy`	특정 정책을 직접 언급. 기본 검색 설정(하한 0.60), 정책명 보너스로 우대, 재판정 안 함
`current_policy_question`	이미 고른 정책에 대한 후속 질문 → 원문 기반 답변

intent (최대 3개)	읽는 원문 섹션
`eligibility` 지원 대상	`target_criteria`
`benefit` 지원 내용	`benefit`
`application` 신청 방법	`application` + `administrative`
`administrative` 기간·행정	`administrative` + `contact_source`
`required_documents` 필요 서류	`documents` + `application`
`contact` 문의처	`contact_source`
`policy_search` / `policy_overview`	`candidate_summary` / `overview`

분석기는 구어체를 정책 용어로 번역합니다(예: "배고파" → "무료 급식 식사 지원"). 이 번역 결과가 `search_query`로 검색에 쓰입니다. 읽을 섹션(`requested_sections`)은 Gemini가 아니라 **코드가 intent 표로 정합니다.**

7. 검색이 동작하는 순서

1. Gemini **질문 분석** — route, intents, 검색어(구어체 번역), 포함·제외 키워드를 얻음. 429·5xx는 1초, 2초 쉬며 최대 3회 재시도.
2. 로컬 **질문 임베딩** — 분석기 검색어와 **원문 질문**(상황형만)을 각각 Qwen3 임베딩으로 벡터화(서버 CPU). 두 검색어를 쓰는 이유: 띄어쓰기·말투에 따라 분석기 검색어가 흔들려도 정답이 사라지지 않게 하기 위함.
3. DB **범위별 조회** — 지자체는 *시/군/구*(선택한 시·군·구)와 *광역*(시·도 전체 정책), 시·군·구가 없으면 *시/도*, 별도로 *중앙부처*(전국) 범위를 각각 조회(`candidate_summary` 레코드 대상, 범위당 30개).
4. 코드 **나이 필터** — 나이를 입력했고 정책의 나이 조건이 명확(`age_status=exact`)할 때만 최소·최대 나이로 제외. 불명확하면 제외하지 않음.
5. 코드 **점수 계산** — 점수 = (1 − 거리) + 범위 가산점 + 정책명 보너스 + 키워드 가산 − 제외 감점.

6 코드 **거리 하한** — 거리가 하한보다 먼 후보는 제외(계약서 규칙: 관련 없으면 억지로 채우지 않음). 단, 정책명 단어가 질문에 모두 있으면 거리와 무관하게 남김.

▼

7 코드 **정렬·분리** — 두 검색어의 결과를 합쳐 정책별로 점수가 높은 후보 하나만 남기고, 지자체/중앙부처를 섞지 않고 각각 점수순 정렬.

▼

7-1 코드 **빈 결과 구제** — 상황형 질문이 0건이면 하한을 0.60으로 **한 번만** 완화해 다시 검색(1위가 0.56처럼 하한을 아슬아슬하게 넘은 경우). 구제된 후보는 "관련도 낮음"으로 가려지며, 무의미 질문은 완화해도 빈 결과.

▼

8 Gemini **AI 재판정**(상황형 질문, 기본 켜짐) — 범위별 후보 12개를 한 번에 판정(8절). 실패하면 거리순 후보 전체로 대체.

값	설정	설명
거리 하한	기본 0.60 / 상황형 0.55	평가 질문 280개 실측: 정답의 99.2% 보존, 무의미 질문 7개 모두 빈 결과. 상황형은 더 엄격(`search_profiles.py`)
범위 가산점	시군구 +0.60 / 광역·시도 +0.35 / 중앙 +0.10	상황형 질문은 비율 0(거리 중심 정렬) — 먼 시군구 정책이 가까운 광역 정책보다 위로 가는 문제를 막음
정책명 보너스	최대 +0.5	정책명 단어(일반어 제외)가 질문에 있는 비율에 비례. 0.5(전부 일치)면 거리 하한 예외
키워드 가산·감점	정책명 ±0.10, 본문 ±0.05	분석기의 포함/제외 키워드, 최대 3개까지. "지원·청년·사업" 같은 일반어는 무시. 후보를 거르지 않고 순서만 바꿈
후보 풀(재판정)	범위별 12개	판정을 쓰지 않는 경로는 범위별 3개

8. AI 관련도 재판정과 후보 표시

왜 필요한가: 거리만으로는 관련 정책과 무관한 정책이 구분되지 않았습니다(상황형 질문 15개 측정: 관련 중앙값 거리 0.45, 무관 0.57로 분포가 겹침). 그래서 검색된 후보를 질문과 함께 Gemini에 보내 "질문한 사람에게 실제로 도움이 되는 정책인가"를 판정합니다.

1. 후보 12개(범위별)의 정책명과 요약 300자를 질문과 함께 **Gemini 1회 호출**(`seed=0`, 시간 제한 10초).
2. 후보별 관련 여부(true/false)를 받아, 판정이 누락·중복되면 실패로 처리. 5xx는 1회만 재시도, 429는 재시도 하지 않음.
3. 범위별로 **관련 판정 후보(점수순) → 무관 판정 후보** 순으로 재구성하고, 무관으로 본 정책 ID를 `low_relevance_ids`로 응답에 담음. **삭제하지 않습니다.**
4. 판정이 실패하면(한도 초과·시간 초과·형식 오류) 오류 없이 **거리순 후보 전체**를 돌려주고 `relevance_checked=false`.

판정 기준: 질문의 상황·필요와 직접 관련이면 관련, 단어만 비슷하면 무관, 지원 대상이 질문자와 명백히 맞지 않으면 무관(예: 질문에 언급이 없는데 장애인·보훈 전용), 지역은 기준 아님, 확실하지 않으면 무관. 질문·정책 문구 안의 지시문은 따르지 않도록 지시합니다.

프론트의 표시 규칙

상황	화면 동작
판정 성공(`relevance_checked=true`)	`low_relevance_ids`에 든 후보가 "관련도 낮음"
판정 안 함·실패	거리 0.50 초과 후보를 "관련도 낮음"으로 간주(백엔드 `SITUATION_CONFIDENT_DISTANCE`와 같은 값)
정책명을 말한 질문(상황형 아님)	거리와 상관없이 모두 일반 후보로 표시
관련 후보가 있을 때	관련 후보 뒤에 관련도 낮음 후보가 배지(점선 테두리 + 안내문)와 함께 이어지고, 3개씩 이전/다음
관련 후보가 하나도 없을 때	"관련도가 높은 정책을 찾지 못했어요" 안내 + "관련도가 낮은 후보 N건 보기" 버튼(누르면 같은 방식으로 펼침)
한쪽 그룹만 비었을 때	다른 그룹에 결과가 있다는 안내 문구로 구분

9. 후속 질문과 답변 (자주 묻는 질문 버튼)

정책을 고르면 이후 질문은 검색이 아니라 `POST /policies/{id}/ask`로 갑니다. 두 가지 경로가 있습니다.

경로	동작	Gemini
버튼 질문	요청에 `intent`(예: `eligibility`)를 함께 보냄 → 서버가 **질문 분석을 건너뛰고** 해당 유형의 섹션만 읽어 답변. 항상 같은 섹션을 읽어 결과가 안정적	1회(답변)
직접 입력	질문 분석 → 현재 정책 질문이면 분석이 정한 intent·섹션으로 답변. 정책과 무관하다고 분류되면 `answer=null`(화면은 "다른 정책 찾기" 안내), 분류가 애매하면 "확인할 수 없음" 답변(`grounded=false`)	2회(분석+답변)

답변은 이렇게 만들어집니다

1. DB에서 **선택 정책의 원문 섹션**만 가져옴(overview, target_criteria, benefit, application, documents, contact_source, administrative).
2. 질문 유형이 요구하는 섹션만 골라 프롬프트에 넣고, 유형별 **[답변 구성]** 안내를 함께 전달.
3. Gemini가 원문만 근거로 답변·근거 구절·사용 섹션·grounded를 JSON으로 반환(`temperature=0`).
4. 코드가 검증: 실제 원문에 없는 근거 구절은 버리고, 근거가 없으면 `grounded=false`. 첨부파일·누리집·법령 링크는 **AI가 아니라 코드가 원문에서 추출**(다운로드/바로가기 버튼).

질문 유형(버튼)	답변 구성(원문에 있는 항목만)
지원 대상	누가 대상인가(연령·거주지·소득·재산·취업/학적 상태) → 제외·우선 대상 → 신청 전 확인할 점
지원 내용·금액	무엇을 지원하나 → 금액·횟수·기간·한도 → 지급 방식·조건
신청 방법	어디서·어떻게 신청 → 절차 순서 → 신청 시기
신청 기간	접수 기간 → 사업 기간·마감 → 상시·연중 여부
필요 서류	제출 서류 목록 → 서식·첨부파일 → 발급처·유의사항
문의처	문의 기관·부서 → 전화·누리집 → 접수 장소

> 원칙: 원문에 없는 금액·나이·기간·서류를 만들지 않고, 자격을 확정하지 않으며("예비 안내"), 확인되지 않는 항목은 항목마다 반복하지 않고 한 줄로 묶어 "제공된 공식 자료에서는 확인되지 않습니다"라고 안내합니다. 직접 입력한 질문도 분석기가 정한 intent로 같은 구성을 씁니다.

10. API 키(BYOK)·보안·오류 처리

사용자 본인 키(BYOK)

- 사용자가 화면의 "API 키"에 **본인 Gemini 키**를 입력합니다. 기본은 탭을 닫으면 사라지는 `sessionStorage`, "기억하기"를 고르면 `localStorage`에 저장합니다.
- 키는 검색·질문 요청의 헤더 `X-Gemini-Api-Key`로만 전달됩니다(URL·본문·로그에 넣지 않음). 서버는 **요청마다** 그 키로 Gemini 클라이언트를 만들어 쓰고 **저장하지 않습니다.**
- 서버 `.env`의 키로 대신 처리하는 폴백은 **기본 꺼짐**입니다(`WELFAREONE_ALLOW_SERVER_KEY=true`일 때만). 공개 배포에서 운영자 키가 소진되지 않게 하기 위함입니다.
- 로그에는 오류 종류만 남기고 질문·키·응답 본문은 남기지 않습니다. 실제 키 값은 저장소·`.env.example`에 넣지 않습니다.

오류 응답과 화면 안내

오류는 항상 `{"error": {"code", "message", "details"}}` 형식이며, 사용자에게 보이는 한국어 문구는 프론트(`constants/text.ts`)가 만듭니다.

HTTP	code	언제	화면 동작
401	`API_KEY_REQUIRED`	키가 없고 서버 키 폴백이 꺼짐	키 입력창 자동 열림
401	`INVALID_API_KEY`	키 형식이 틀리거나 Gemini가 거부(401/403)	키 입력창 열림(재입력)
400	`INVALID_REGION`	목록에 없는 지역	지역 재선택 안내
422	`INVALID_REQUEST`	빈·500자 초과 질문, 허용 밖 `intent` 등	입력 확인 안내
404	`POLICY_NOT_FOUND`	없는 정책 ID	다른 정책 선택 안내
429	`LLM_RATE_LIMITED`	Gemini 한도 초과	잠시 후 재시도 안내
502	`LLM_RESPONSE_INVALID`	AI 응답 처리 실패	재시도 안내
500	`DATABASE_ERROR` / `INTERNAL_ERROR`	DB 오류 / 예상 못한 오류	재시도 안내
-	`NETWORK_ERROR` / `TIMEOUT`	(프론트) 서버 연결 실패 / 30초 초과	상태 확인 안내

실패해도 멈추지 않는 부분

- AI 재판정 실패 → 오류 대신 거리순 후보 전체로 대체(`relevance_checked=false`).
- Gemini 429·5xx → 분석·답변은 자동 재시도(1초, 2초 대기, 최대 3회), 그래도 실패하면 위 오류 코드로 안내.
- 저장소(localStorage) 사용 불가·용량 부족 → 오래된 상담부터 지우며 재시도, 그래도 안 되면 저장만 건너뛰고 화면은 계속 동작.

11. 브라우저에 저장되는 데이터

항목	저장소	내용
상담 기록	localStorage	상담마다 키를 따로 저장(`.conv.<id>`)하고 목록용 요약은 `.index`. 최대 30개 상담, 상담당 메시지 200개. 후보의 개발용 값(metadata·reasons)은 비워 용량 절감. 다른 탭에서 바뀌면 목록 갱신
마지막으로 본 상담	localStorage	새로고침 후 복원
테마	localStorage	시스템/라이트/다크
Gemini API 키	sessionStorage(기본) / localStorage	상담 기록과 별도 키로 저장, 상담 기록에는 절대 포함하지 않음

서버에는 사용자 데이터가 저장되지 않습니다(stateless). 공용 PC에서는 사용 후 기록 삭제를 안내합니다.

12. 데이터(DB) 만드는 과정

1. **수집** — `scripts/download_welfare.py`: 복지로 공공 API에서 목록·상세 XML을 `data/raw`로 내려받음.
2. **변환** — `scripts/export_welfare_jsonl.py`: XML을 읽기 쉬운 JSONL(`data/processed`)로 변환.
3. **적재** — `scripts/import_welfare_chroma_qwen_v3.py`: 정책을 의미 단위 섹션으로 나누고 정책마다 `candidate_summary` 한 건을 만들어 Qwen3 임베딩과 함께 ChromaDB에 저장(원본 XML이 기준).

섹션(`section`)	내용 / 쓰임
`candidate_summary`	정책 요약(정책명·지역·생애주기·개요·대상·지원 내용, 최대 약 450자). **검색 대상**
`overview`, `target_criteria`, `benefit`	개요, 지원 대상·선정 기준, 지원 내용·금액. 답변·원문 보기에 사용
`application`, `documents`, `contact_source`, `administrative`	신청 방법, 서류, 문의처·출처, 기간·수정일·법령

레코드의 메타데이터에는 `policy_id`(예: `local:WLF00006618`), 출처(`source`: local/national), 시·도·시·군·구, 나이 범위와 `age_status` 등이 들어 있고, 검색은 이 값으로 지역·범위를 필터링합니다. **정책 식별은 항상 `policy_id`**입니다. DB 원본과 컬렉션명은 팀 합의 없이 바꾸거나 재적재하지 않습니다(임베딩 모델도 DB를 만든 모델과 같아야 함).

13. 실행·설정·테스트

용도	명령 / 값
백엔드 실행	`uvicorn app.api:app --port 8000` (프로젝트 루트, 시작할 때 모델 로딩으로 시간이 걸림)
프론트 실행	`cd frontend; npm install; npm run dev` (기본 5173, 사용 중이면 Vite가 알려 주는 포트). 실제 연결은 `frontend/.env.local`에 `VITE_USE_MOCK=false`, 기본값은 mock
터미널 프로토타입	`python -m app.main` (서버 키 `GEMINI_API_KEY` 필요)
환경 변수	`GEMINI_API_KEY`(터미널·서버 폴백 전용), `GEMINI_MODEL`, `WELFAREONE_ALLOW_SERVER_KEY`(기본 꺼짐), `WELFAREONE_LLM_RERANK`(`0`이면 AI 재판정 끔)
백엔드 테스트	`python -m unittest discover -s tests` (키·DB 불필요, 131개)
실DB 스모크	`WELFAREONE_REAL_TESTS=1`을 켜고 `python -m unittest tests.test_real_smoke`
프론트 테스트·빌드	`npm test`(39개), `npm run typecheck`, `npm run build`

준비물: Python 패키지(`pip install -r requirements.txt`), 벡터 DB(`data/chroma`), 임베딩 모델(Hugging Face 캐시), 사용자 Gemini 키. `.env`·키·`data/chroma`는 커밋하지 않습니다.

14. 비용·지연, 한계와 다음 작업

Gemini 호출 횟수와 지연

경우	Gemini 호출	측정한 시간(참고)
정책 검색(상황형, 재판정 켜짐)	2회	질문 분석 약 1.4초 + 판정 약 2.4초(후보 12개 기준). 첫 요청은 더 걸릴 수 있음
정책 검색(정책명 질문 등, 재판정 없음)	1회	분석만
버튼 질문	1회	약 1.3~2.6초
직접 입력 후속 질문	2회	분석 약 1.4초 + 답변 약 1.8초
원문 보기·지역 목록·쪽 넘김·기록 관리	0회	즉시

사용자 본인 키를 쓰므로 호출 비용과 한도(429)는 사용자 키마다 다릅니다. 재판정은 `WELFAREONE_LLM_RERANK=0`으로 끌 수 있습니다.

알아 둘 한계

- **평가 근거가 AI 판정**: 관련도 정답을 사람이 아닌 AI가 판정했고 질문도 15개뿐이라 수치는 낙관적일 수 있습니다. 사람 라벨링(질문 50개)으로 기준값을 다시 확정할 예정입니다.
- **집단 전용 정책**: 장애인·보훈 전용 등은 AI 판정이 "관련도 낮음"으로 뒤로 보냅니다. "대상 한정" 표시 방식은 논의가 필요합니다.
- **원문이 없는 항목**: 서류·신청 기간이 DB에 없는 정책은 해당 버튼에서 "확인되지 않습니다"가 나옵니다.
- **검색 대상**이 요약(`candidate_summary`) 한 건이라 대상·조건 문구가 잘리면 매칭이 약해집니다.
- **정책 외 질문**은 분리된 경로가 없어 "확인할 수 없음" 답변으로 처리됩니다(기존 동작 유지).
- 모든 AI 단계가 Gemini에 의존해 모델이 바뀌면 결과가 조금 달라질 수 있습니다(seed 고정·폴백으로 완화).

다음 작업 후보

- 사람 라벨링으로 거리 기준(0.55/0.50), 후보 수(12), 재판정 켜짐 기본값 확정 (`docs/labeling/labeling_guide.pdf`)
- 검색어의 일반어("지원", "청년")로 같은 정책이 반복되는 문제, 섹션 단위 검색 확장
- 자격 판정(`eligibility_service`), 서류 체크리스트, 신청서 초안, 대화 상태·LangGraph (`docs/DIRECTION_C_PLAN.md`)
- Reranker 모델 병행은 실험에서 상황형 질문 개선이 확인되지 않아 보류
- API 변경(`relevance_checked`, `low_relevance_ids`, `/ask`의 `intent`)의 계약서 반영 합의 (`docs/contract_update_proposal.md`)

부록. 용어 풀이

용어	뜻
route / intent	route = 질문이 어떤 처리 경로로 가는지(상황 검색 / 정책명 / 후속 질문). intent = 질문의 목적(대상·금액·신청 방법 등)
임베딩 / 거리	글을 숫자 벡터로 바꾼 것. 두 글의 거리가 가까울수록(0에 가까울수록) 의미가 비슷함(코사인 거리)
RAG	검색(Retrieval)으로 근거 원문을 찾아 그 원문만으로 AI가 답변(Generation)하게 하는 방식
BYOK	Bring Your Own Key. 사용자가 본인 API 키를 직접 넣어 쓰는 방식
재판정(rerank)	검색된 후보를 AI가 다시 보고 관련 여부를 판단해 순서를 바로잡는 단계
grounded	답변의 핵심이 제공된 원문으로 뒷받침되는지 여부. false면 화면에 경고
seed / temperature	AI 응답의 무작위성을 줄여 같은 입력에 같은 결과가 나오게 하는 설정
폴백(fallback)	한 단계가 실패했을 때 오류 대신 더 단순한 방식으로 대체하는 동작
stateless	서버가 사용자 상태를 저장하지 않고 요청마다 필요한 정보(지역·나이·선택 정책)를 받는 구조
MSW / mock	백엔드 없이 브라우저에서 가짜 API 응답을 만들어 화면을 개발·검증하는 도구